

Developer Onboarding Guide — NextGen ERP

Welcome to the **NextGen Interior & Waterproofing ERP** development team! This onboarding guide is compiled to help you understand the architectural guidelines, setup procedures, and code standards required to contribute to the platform.

SECTION 1: Project Overview for New Developers

The platform manages multi-warehouse material inventories, bulk procurement cycles, wholesale/retail billing registers, client site issues, and double-entry accounting files for specialized construction operations in Nepal.

- **Backend Architecture:** React Server Components (RSC) and serverless server actions, operating direct type-safe data pipelines.
 - **Relational Integrity:** Prisma ORM wrapping a Supabase PostgreSQL instance under strict composite index bounds.
 - **Core Technology Stack:**
 - Next.js 16.0.0 (App Router paradigm)
 - TypeScript 5.x
 - Prisma ORM 7.8.0
 - PostgreSQL 16
 - NextAuth.js v5 (Beta)
 - TailwindCSS 4.0
 - Zod Schemas
 - Resend API (OTP recover mailings)
 - **Code Repository:** Hosted on private GitHub repositories.
 - **Deployment Environments:** Automatic deployments linked to Vercel (Staging / Production).
-

SECTION 2: Local Development Setup

Follow these numbered steps to clone, configure, and boot the ERP application locally:

1. Verification of Prerequisites:

Ensure your development workstation has the following base binaries installed:

- **Node.js:** Version 20.x or higher (node -v should return >=v20.0.0).
- **npm:** Version 10.x or higher (npm -v).
- **Git:** Installed and configured (git --version).
- **PostgreSQL:** A local PostgreSQL 16 database running if you want to develop offline (otherwise, you can connect directly to a Supabase dev slot).

2. Clone the Codebase:

```
git clone https://github.com/nextgen-interior/nextgen-erp.git
cd nextgen-erp
```

3. Dependency Assembly:

Run the `install` script. This compiles binary bindings and sets up local modules:

```
``bash
npm install
```

Expected Output: added XXX packages, and audited YYY packages in Zs. 0 vulnerabilities found.

4. Environmental Configuration:

Copy the environment variables template and configure your local overrides:

```
cp .env.example .env.local
```

Action required: Open .env.local and enter your local or sandbox DATABASE_URL, NEXTAUTH_SECRET (generate with openssl rand -base64 32), and Resend keys.

5. Database Schema Construction:

Deploy the Prisma schema definition directly to your database instance:

```
npx prisma db push
```

Expected Output: Prisma schema loaded... Databases are now in sync with your Prisma schema.

6. Sandbox Database Seeding:

Seed your local database with essential lookup files (admin users, default warehouses, product categories, and brand codes):

```
npm run db:seed
```

Expected Output: Seeding completed successfully. Admin account: admin@nextgen.com.np generated.

7. Run the Local Compiler Dev Server:

```
npm run dev
```

Expected Output: Next.js 16.0.0... Ready in Zms. Local: http://localhost:3000

Open your browser to <http://localhost:3000> and verify you can see the login screen!

SECTION 3: Project Structure Explained

The system boundaries are organized under a standard Next.js multi-tier architecture.

```
nextgen-erp/
```

```
├── src/
```

```
│   ├── app/           Next.js App Router (coordinates client routing & page layouts)
```

```
│   │   ├── (auth)/    Public verification portals (forgot-password, login screens)
```

```
│   │   ├── (dashboard)/ Protected business dashboards (sales, accounting, stock pages)
```

```
│   │   └── api/        External REST API route handlers (Excel streams, lookup API)
```

```
│   └── modules/        Domain Business Logic Layer (completely isolated by module directory)
```

```
│       ├── sales/      Manages billing sheets, credit accounts, returns actions
```

```
│       │   ├── actions.ts Server Actions (mutations, RPC calls)
```

```
│       │   ├── queries.ts Read Operations (findMany, findUnique)
```

```
│       └── types.ts     TypeScript definitions and Zod schemas
```

other modules/	Identical structure pattern across the other 9 modules
components/	Reusable Presentation Layer Components
layout/	Persistent shell templates (Sidebar navigations, page headers)
shared/	Base visual widgets (buttons, modals, search bars)
[module]/	Module-specific UI views (Invoices table, Projects card)
lib/	Infrastructure Services Singleton Utilities
db.ts	Prisma Client instance initializer
utils.ts	Generic formatting methods (formatNPR, nextCode generation)
settings-store.ts	Business Settings service
nepali-date.ts	Dual calendar converter adapters (AD to BS)
prisma/	ORM Schema Management
schema.prisma	Prisma relational mapping schema
seed.ts	Lookup seeder logic
scripts/	Operational utilities (backups, docker-compose)
.github/workflows/	GitHub Actions automated pipeline workflow files

SECTION 4: Coding Conventions

4.1 Server Action Pattern (Mutations)

All Server Actions modifying database states must use the following standard structure:

```
"use server";
```

```
import { auth } from "@auth";
import { getDb } from "@lib/db";
import { revalidatePath } from "next/cache";
import { createSchema } from "../types";
```

```
export async function createFeatureAction(data: unknown) {
  // 1. Authenticate user session outside transaction
  const session = await auth();
  if (!session?.user?.id) {
    return { success: false, error: "Unauthorized access", code: "UNAUTHORIZED" };
  }
```

```
  // 2. Validate parameter input types using strict Zod schemas
  const parsed = createSchema.safeParse(data);
  if (!parsed.success) {
    return { success: false, error: "Validation failed", code: "VALIDATION_ERROR" };
  }
```

```

}

const db = await getDb();

try {
  // 3. Coordinate multi-row queries inside an atomic transaction block
  const result = await db.$transaction(async (tx) => {
    // Execute database queries using transaction client 'tx'
    const record = await tx.myModel.create({ data: parsed.data });

    // 4. Create an audit trail log
    await tx.auditLog.create({
      data: {
        userId: session.user.id,
        action: "CREATE",
        module: "MY_MODULE",
        recordId: record.id,
        newValues: record as any,
      }
    });

    return record;
  });

  // 5. Revalidate cache segments to render fresh items in real-time
  revalidatePath("/my-module");
  return { success: true, data: result };
} catch (error: any) {
  console.error("Mutation failed", error);
  return { success: false, error: error.message || "Database update failed", code:
"DATABASE_ERROR" };
}
}

```

4.2 Query Pattern (Reads)

- **Data Deserialization:** PostgreSQL Decimal types do not serialize natively across Server Actions to Client Components. All queries returning database results must wrap their payloads in `serializeForClient()` (defined in `@/lib/utils`) before sending them to the client.
- **Rolling Pagination:** Always enforce pagination limits (default 20 records) using `take` and `skip` arguments inside Prisma lookups to optimize database processing.

4.3 Component Pattern

- **Component Separation:** Keep page wrappers as **Server Components** to fetch initial data directly from queries, passing down serialized models to **Client Components** for interactivity.
- **Loading Layouts:** Always provide a matching loading.tsx skeleton layout page inside App Router routes to guarantee high perceived loading speed.

4.4 Rules for Financial Values:

- **Never Use Float/Number Types for Money:** Floating-point values introduce rounding errors. Always declare price/value fields as Decimal in schema.prisma.
- **Formatting:** Always display financial amounts using the formatNPR() utility, formatting values into Nepali lakhs representation with standard NPR symbol headers.

4.5 Calendar & Date Rules:

- **Gregorian Internals:** Always save dates inside database tables as standard Gregorian UTC DateTime values to support standard queries.
- **Nepali Presentation:** Render dates in Bikram Sambat (BS) format using local date utilities (e.g. DualDateDisplay and DualDatePicker inputs), showing BS as the primary visual display and AD as a secondary reference.

SECTION 5: Adding a New Feature (Step-by-Step Guide)

If you are tasked with adding a new functional module (e.g., a "Quotations" catalog), follow these steps:

1. **Prisma Model:** Open prisma/schema.prisma and define the model model Quotation { ... } with all target indexes (@@index).
2. **Schema Push:** Run npx prisma db push to generate typescript type definitions and update your database structure.
3. **Module boundaries:** Create the module folder /src/modules/quotations/.
4. **Zod Schemas:** Inside /quotations/, create types.ts defining input parser Zod validation schemas.
5. **Database Reads:** Create /quotations/queries.ts containing search and filter reads.
6. **Server Mutations:** Create /quotations/actions.ts containing transaction mutations and audit loggers.

7. **UI Views:** Create the folder `/src/components/quotations/` and build presentation widgets (e.g. `QuotationTable.tsx`).
 8. **App Router Route:** Create `/src/app/(dashboard)/quotations/page.tsx` as a Server Component, load lookups, and render components.
 9. **Visual navigation:** Add the quotation page link to the sidebar layout file `/src/components/layout/Sidebar.tsx`.
 10. **Security permissions:** Update your RBAC authorization logic inside server actions to declare which roles can read/write quotations.
 11. **Static Verification:** Execute `npx tsc --noEmit` and fix any compiler warnings.
 12. **Test Compilation:** Run `npm run build` locally to confirm the production bundle compiles.
 13. **Git Commit:** Commit changes to your feature branch and push to open a Pull Request.
-

SECTION 6: Testing Checklist

Before opening a PR or merging into staging, verify that all checkboxes pass:

Pre-commit Verification:

- ☐ `npx tsc --noEmit` executes with zero compilation errors.
- ☐ `npm run build` completes successfully.
- ☐ Direct validation testing on the feature has been verified manually in Chrome and Safari.
- ☐ Browser developer console shows zero warning logs or failed runtime errors.

Schema Modifications:

- ☐ `npx prisma validate` passes check.
- ☐ Schema changes have been verified locally using `npx prisma db push`.
- ☐ New master mock tables are mapped to `prisma/seed.ts`.
- ☐ Structural migrations are backward compatible, preventing any historical data loss on live tables.

Financial Bookkeeping logic:

- ☐ 13% VAT tax calculations are verified to be mathematically accurate.
- ☐ Ledger entry changes trigger matching Debit/Credit ledger entries, keeping company assets

balanced.

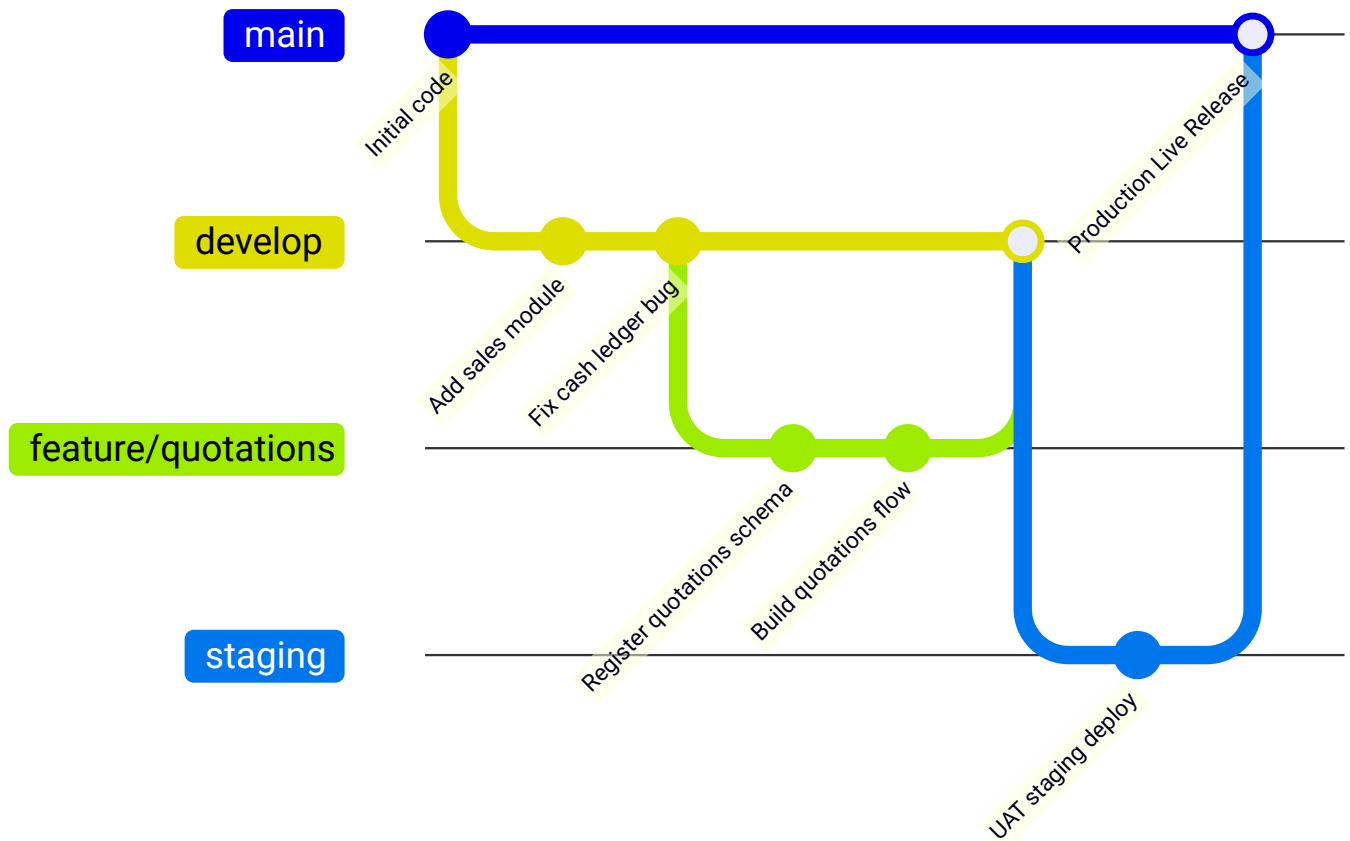
- ☐ Invoices are blocked immediately if warehouse quantities are lower than required, keeping inventory stock accurate.
 - ☐ General Trial Balance debits and credits sum up to a perfect zero balance.
-

SECTION 7: Troubleshooting Common Developer Issues

- Issue: Compilation error: Cannot find module '@/...' or import errors.
 - **Resolution:** TypeScript paths require aliases. Check case sensitivity on imports (Linux staging servers are case-sensitive, unlike macOS/Windows sandboxes).
 - Issue: Prisma P2002 Unique Constraint violation on unique invoice/receipt code creation.
 - **Resolution:** The nextCode() generator utility hit a key collision. Check the database row count for index prefixes and clear conflicting test rows.
 - Issue: Next.js Runtime: Error: Plain objects are not serializable...
 - **Resolution:** PostgreSQL Decimal numbers or JS Date objects are being returned directly from database calls. Wrap your return query payload with serializeForClient(data) before passing to Client Components.
 - Issue: Current session returns undefined inside a Server Action transaction block.
 - **Resolution:** NextAuth session validation blocks cannot execute inside asynchronous Prisma transaction streams. Call const session = await auth() outside and before starting your \$transaction pooler block.
-

SECTION 8: Git Workflow

All developers must follow this branch hierarchy. Pushing directly to main or staging is strictly blocked by repository security rules.



Branches & Rules:

- **develop**: The primary daily integration branch. All feature branches merge here.
- **feature/*** or **fix/***: Developers check out local feature branches from develop and open Pull Requests back to develop.
- **staging**: Pre-production environment. Only merges from develop are allowed here, which triggers the automated Vercel UAT staging build.
- **main**: The live production application. Merges are allowed ONLY from staging after UAT sign-off.

SECTION 9: GitHub Secrets Required

To support automated staging and production deployment workflows, the repository requires the following keys configured under **GitHub Settings -> Secrets and Variables -> Actions**:

1. **DATABASE_URL**: The connection string referencing your primary Supabase database instance. Used to run migrations during CD runs.
2. **VERCEL_TOKEN**: Personal access token from your Vercel Account. Authorizes GitHub actions to trigger Vercel compilation builds.

3. **VERCEL_PROJECT_ID**: The identifier of your NextGen ERP project in the Vercel workspace.
4. **VERCEL_ORG_ID**: The Vercel organization/user identifier.

For variables rotation procedures and coordinate values, refer to the [Operations & Deployment Runbook](#).